



Syed Babar Ali
School of Science and Engineering

CS5305/CS621

Programming Assignment 1

Due Date: February 22nd, 2026

Introduction

In this assignment, you are required to acquire and analyze NYC **Yellow Taxi** Trip Record data and using various Apache spark APIs.

The assignment further introduces Medallion Architecture, a pattern to organize data in Data lake/Lakehouse environments, and develops the process of incrementally refining the data through its various layers.

You will sign up for a free databricks account where you will create your notebook and perform all the tasks.

Deliverable: You will need to submit your final notebook which clearly shows all the tasks and their output. Name your notebook as: **<roll_number>-pa1.ipynb**

Medallion Architecture

The Medallion Architecture is a data design pattern used to logically organize data in a Lakehouse. The goal is to incrementally improve the structure and quality of data as it flows through each layer.

The Three Layers:

1. **Bronze (Raw):** The landing zone. Data is stored in its raw format, often including metadata like the source file name and ingestion timestamp.
2. **Silver (Validated):** The "cleansed" zone. Data is filtered, joined, and standardized to provide an enterprise-wide view of the entities.
3. **Gold (Aggregated):** The "business-ready" zone. Data is aggregated and modeled into star schemas or reporting tables for final consumption.

Unity Catalog

Unity Catalog is Databricks' **centralized governance layer** for all data and AI assets.

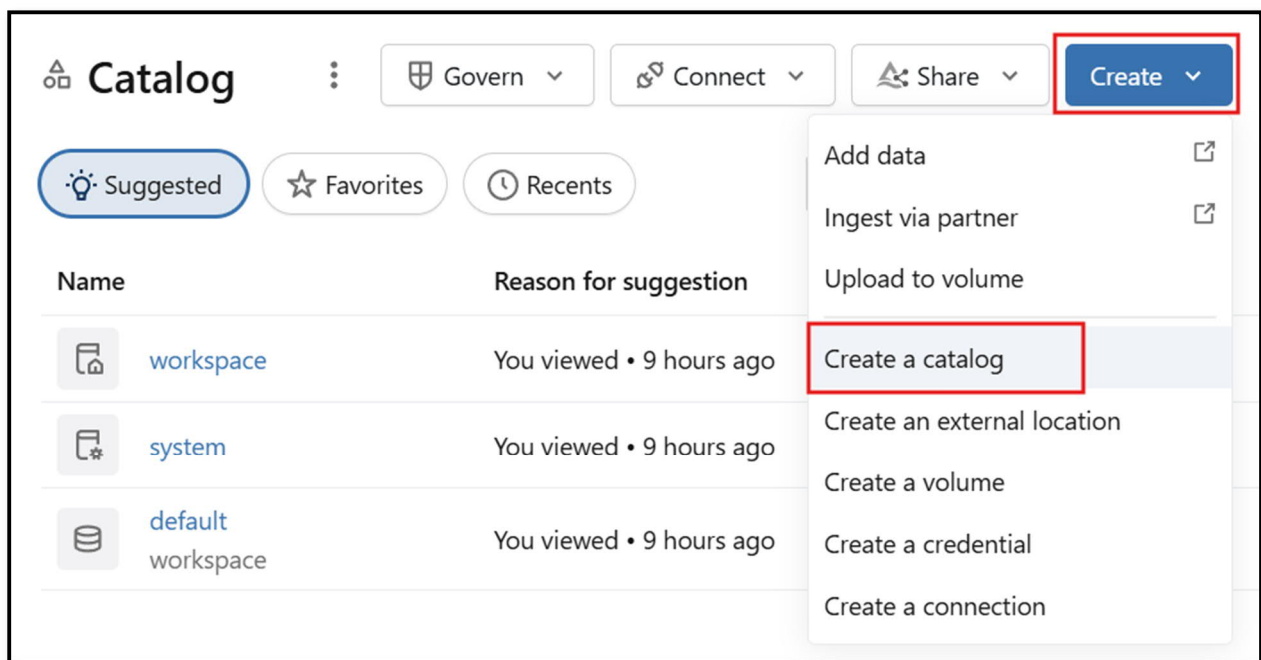
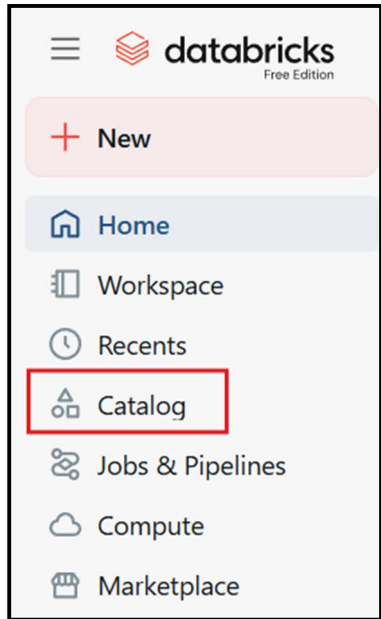
It provides a **single source of truth** for managing:

- Data (tables, views, files)
- ML models
- Notebooks, functions, dashboards

Key Capabilities

- **Centralized metadata management** – catalogs → schemas → tables
- **Fine-grained access control** down to columns & rows
- **Automatic data lineage tracking** end-to-end
- **Secure table/data sharing via Delta Sharing**

Unity Catalog is access from the catalog menu item in databricks:



Create a new catalog

×

A catalog is the first layer of Unity Catalog's three-level namespace and is used to organize your data assets. [Learn more](#)

Catalog name*

PA1

Type*

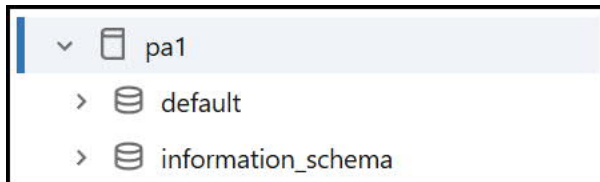
Standard

Storage location

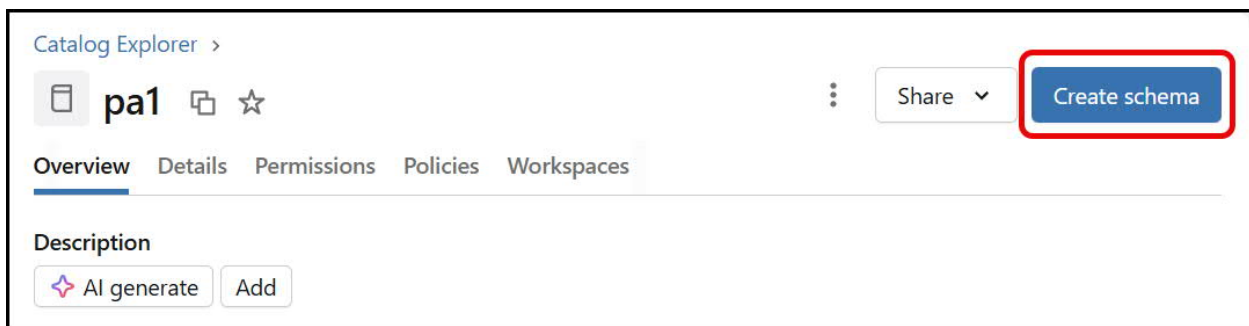
☒ Use default storage

Cancel

Create



When you create a new catalog, it automatically creates two schemas as shown above (default, and information_schema). We need to implement our medallion architecture in the unity catalog now. For that, we'll create three new schemas, bronze, silver, and gold:



Create a new schema

×

A schema is the second layer of Unity Catalog's three-level namespace and organizes tables and views. [Learn more](#)

Schema name*

bronze

Storage location

Cloud storage location used for managed tables and volumes in this catalog. If not specified, it defaults to the metastore root location.

 Select external location

sub/path

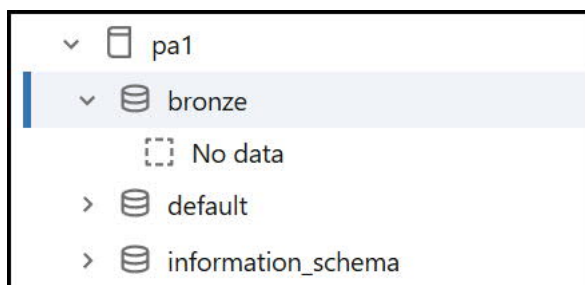
[Create a new external location](#)

Comment

Cancel

Create

After clicking create, you will see new schema named 'bronze' appear under your catalog:



Repeat the steps to create silver and gold schemas as well.

Once you are familiar with unity catalog, delete your PA1 catalog. You will be doing that from your notebook as part of assignment.

Create your notebook now under your databricks workspace and perform all the required tasks.

1. Task: Unity Catalog Setup

Create unity catalog with the naming convention <roll_number>-pa1. Create bronze, silver and gold schemas under it.

Reference:

<https://learn.microsoft.com/en-us/azure/databricks/catalogs/create-catalog>

2. Task: Setup Bronze Layer (Raw Ingestion)

Goal: Ingest the raw Parquet files into a Delta table without modifications.

Dataset Source: [Official NYC TLC Trip Record Data](#)

- **Target Data:** Yellow Taxi Trip Records
- **Timeframe:** 6 consecutive months from any year
 1. Download the files to a temporary directory first. You need to perform this download within your notebook using shell and/or dbutils commands.
 2. Load all the downloaded Parquet files into a single Spark DataFrame and add a new column ingestion_timestamp using the current time.
 3. Write this DataFrame to a Delta table taxi_trips under bronze schema.
 4. Show the transaction log of your table.
 5. Demonstrate "Time Travel" by querying the table using a previous version number or timestamp to show the state before all files were fully merged.

Tips/Reference:

- <https://learn.microsoft.com/en-us/azure/databricks/ingestion/file-upload/download-internet-files>
-

3. Task: Setup Silver Layer (Cleansing & Refining)

Note: Data Dictionary for the Yellow Taxi Trips can be retrieved from the below link:

https://www.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_yellow.pdf

Goal: Clean the data and ensure it is ready for analysis.

- Perform data quality checks: Filter out records where trip_distance is zero or negative, and where total_amount is less than 0.

- Standardize the data types for pickup and drop-off times and calculate a new column `trip_duration_minutes`.
 - Simulate a data update. Pick 10K random records and change their `payment_type`. Use the `MERGE INTO` command to update these records in your Silver table while maintaining ACID consistency.
 - Add a new dummy column (e.g., `driver_notes`) to your Silver table. Configure your write operation to allow for **Schema Evolution** so the table schema updates automatically. Write the mapped value of `RatecodeId` to this column.
 - Create delta table in your silver layer with data filtered based on vendor id. Name the table as `nyc_yellowtaxi_<vendor_id>`. Show percentage of data belongs to each vendor in your notebooks.
 - What is the ratio of tip amount to the fare paid?
 - Out of total taxis, what percentage of taxis stored the data in the vehicle memory before forwarding the saved batch to the server?
-

4. Task: The Gold Layer (Business Insights)

Goal: Create high-value, aggregated tables for the business.

- Create a table under gold schema that has the **Average Fare Amount** and **Total Trip Count** per `PULocationID` (Pickup Zone) for each of the months. Name the table `avg_fare`.
 - Join your taxi data with the [Taxi Zone Lookup Table](#) to replace IDs with actual Borough and Zone names. Name your table as
 - Based on the pickup location and drop of locations, group and aggregate data for each to-from location (total fare paid, total tips paid, total tips, airport fee, etc). Then look up the IDs from the respective data dictionary and replace location ID with the actual location name. Replace all IDs with their respective values. Name the table as `trip_aggregates_with_locations`.
 - o You will need the [Taxi Zone Lookup Table](#) to replace IDs with actual location and zone name.
 - **Delta Lake Performance Optimization:** Run the `OPTIMIZE` command on your Gold table and apply **Z-ORDER** on a column to speed up time-based queries.
 - o Clarify in the notebook which column you picked and why.
 - o Demonstrate query timings updates before and after performing z-ordering.
-